UNIVERSIDAD AUTONOMA DE CHILE

Introducción a la Robótica

Diseña, imprime y programa tu propio robot futbolista

Guía paso a paso: diseño CAD en OnShape, impresión 3D en PLA, electrónica con Arduino UNO y módulo L298N, programación en C++ y control por Bluetooth desde el celular.

Eduardo Fuentes Fierro
Ingeniero Civil Mecánico
Magíster en Ciencias de la Ingeniería

Primera edición · 2026

Introducción a la Robótica
Diseña, imprime y programa tu propio robot futbolista

Primera edición, 2026.

Eduardo Fuentes Fierro
Ingeniero Civil Mecánico
Magíster en Ciencias de la Ingeniería
Universidad Autónoma de Chile

Esta obra se distribuye bajo licencia
Creative Commons Atribución-NoComercial-CompartirIgual 4.0 Internacional (CC BY-NC-SA 4.0)

Eres libre de copiar, distribuir y adaptar este material citando la fuente, sin fines comerciales y compartiendo las obras derivadas bajo la misma licencia.

Los archivos del proyecto (código Arduino, modelos STL y este libro en PDF) están disponibles para descarga libre en el repositorio del libro:
`https://github.com/Edward3th/UA_Robotica`

Compuesto con LaTeX (LuaLaTeX, clase KOMA-Script).
Tipografías: TeX Gyre Pagella, TeX Gyre Heros y TeX Gyre Adventor.

El logotipo de la Universidad Autónoma de Chile se reproduce con fines identificatorios. Los nombres de productos y marcas mencionados (Arduino, OnShape, Ultimaker Cura, MIT App Inventor) pertenecen a sus respectivos titulares y se citan con fines educativos.

Índice general

0 Prólogo: ¿por qué construir un robot? **1**
0.1 ¿Qué es la robótica? 1
0.2 La robótica hoy 1
0.3 Qué construirás en este libro 2
0.4 Cómo usar este libro 3

1 Materiales y software **5**
1.1 Lista de materiales 5
1.2 Software que vas a instalar 6

2 Diseño CAD en OnShape **9**
2.1 ¿Qué es el CAD paramétrico? 9
2.2 La interfaz de OnShape 9
2.3 Croquis y restricciones 9
2.4 Operaciones básicas 10
2.5 Manos a la obra: diseña tu chasis 11
2.6 Exportar a STL 11
2.7 El chasis del autor 12

3 Impresión 3D **15**
3.1 Manufactura aditiva: qué es y por qué FDM 15
3.2 El material: PLA 15
3.3 Diseño para impresión 3D 16
3.4 Laminando en Cura 16
3.5 Manos a la obra: imprime tu chasis 17
3.6 Fallas típicas 17

4 Electrónica **19**
4.1 Conceptos base 19
4.2 Descripción de componentes 19
4.3 Por qué necesitas un puente H 20
4.4 Servomotores y ancho de pulso 20
4.5 Alimentación 21
4.6 Manos a la obra: cableado y ensamble 21

5 Programación **25**
5.1 ¿Qué es programar? 25
5.2 Estructura de un programa Arduino 25
5.3 De un motor a dos: funciones y tracción diferencial 26
5.4 Servos y la función de pateo 27
5.5 Manos a la obra: tu rutina de prueba 28
5.6 Depuración con el Monitor Serie 28

6 Comunicación **31**
6.1 ¿Qué es un protocolo, y por qué Bluetooth? 31
6.2 El módulo HC-06 y SoftwareSerial 31
6.3 El protocolo de comandos del robot 32

6.4 MIT App Inventor: construye la app 33
6.5 Manos a la obra: empareja y controla tu robot 34

7 Epílogo 37
7.1 Reglas para jugar 37
7.2 Lo que aprendiste 37
7.3 Hacia dónde seguir 37

A Diagrama de conexiones 39

B Guía de resolución de problemas 41

C Archivos descargables 43

0 Prólogo: ¿por qué construir un robot?

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es la robótica y por qué es un campo interdisciplinario.
- Reconocer robots que ya te rodean y situar el fútbol robótico como problema de aprendizaje.
- Saber qué construirás exactamente a lo largo de este libro.
- Conocer cómo está organizado el libro y cómo sacarle provecho.

0.1 ¿Qué es la robótica?

Un robot no es una sola disciplina: es el punto donde varias se encuentran. La **mecánica** le da un cuerpo capaz de moverse (ruedas, ejes, engranajes). La **electrónica** le entrega lógica y control (motores, servos, sensores). La parte **eléctrica** resuelve cómo llega la energía desde la batería hasta cada componente sin que nada se queme en el camino. Y la **informática**, a través de un programa, decide *qué* hace el robot y *cuándo* lo hace.

Ningún robot funciona si falla una sola de estas cuatro capas. Un chasis perfecto con una batería mal conectada no se mueve. Un motor bien alimentado sin código no sabe qué hacer. Por eso la robótica se aprende mejor construyendo: cada capítulo de este libro resuelve una capa a la vez, y solo al final las cuatro conviven en la misma máquina.

NOTA

No necesitas saber nada de electrónica, CAD ni programación para empezar. Esta es una guía práctica que asume que no tienes experiencia previa.

0.2 La robótica hoy

Ya convives con robots todos los días, aunque no siempre los llames así: el brazo que suelda carrocerías en una fábrica de autos, el robot aspirador que esquiva los muebles de una casa, el dron que entrega un paquete. Todos comparten la misma estructura de fondo que vas a construir en este libro: sensores o entradas que informan del entorno, un controlador que decide, y actuadores que ejecutan esa decisión sobre el mundo físico. La Figura 0.1 muestra un ejemplo de cada contexto: la industria, el hogar y la competencia.

Para aprender robótica hace falta un problema concreto, no solo teoría. La **RoboCup** —la competencia internacional de fútbol robótico— es un buen problema porque obliga a resolver movimiento, control y comunicación a la vez, con una meta simple de entender: mover un balón hacia un arco. No vas a competir en la RoboCup, pero vas a construir el mismo tipo de máquina que ahí compite, a una escala pensada para tu primer robot.



Figura 0.1. Robots presentes en distintos contextos: industria, hogar y competencia. Fotografías: Phasmatisnox (CC BY 3.0), Air fans (CC0) y Eneas De Troya (CC BY 2.0); montaje y recortes propios.

0.3 Qué construirás en este libro

A lo largo de estos capítulos vas a diseñar, imprimir, cablear, programar y controlar un robot futbolista: un vehículo de **tracción diferencial** —dos motores DC independientes, uno por lado— con dos servomotores que actúan como paletas laterales, al estilo de un pinball, para golpear un balón. Lo vas a controlar de forma inalámbrica desde una aplicación que tú mismo vas a construir para tu celular. La Figura 0.2 muestra el resultado.

[IMAGEN PENDIENTE]

`cap00-02-robot-terminado`

Figura 0.2. El robot futbolista terminado: vista general con sus componentes principales.

CONSEJO PRÁCTICO

Si en algún capítulo no ves de inmediato para qué sirve lo que estás construyendo, vuelve a la Figura 0.2. Cada pieza del libro —cada motor, cada línea de código, cada conexión Bluetooth— existe para que este robot termine funcionando.

0.4 Cómo usar este libro

El libro está organizado como una secuencia de construcción, no como un manual de referencia: cada capítulo se apoya en el anterior y te acerca un paso más al robot terminado. Vas a encontrar siete elementos recurrentes que marcan el ritmo de la lectura:

- Una caja de **objetivos** al abrir cada capítulo, como la de arriba.
- Cajas de **nota** con información complementaria.
- Cajas de **atención** cuando hay riesgo real de daño a un componente.
- Cajas de **consejo práctico** con atajos y errores comunes.
- Cajas de **manos a la obra** con la tarea concreta que ejecutas.
- Cajas de **antes de seguir, verifica** al cierre de cada sección clave.
- Un **resumen** al final de cada capítulo.

No necesitas ningún conocimiento previo de electrónica, CAD ni programación, ni tampoco herramientas más allá de las que se listan en el Capítulo 1. Lo que sí necesitas es continuar los capítulos en orden: cada uno da por hecho lo que construiste en el anterior.

RESUMEN DEL CAPÍTULO

- La robótica combina mecánica, eléctrica, electrónica e informática: ninguna capa funciona sola.
- Los robots que ya te rodean comparten la misma estructura —sensores, control, actuadores— que vas a construir en este libro.
- Vas a construir un robot futbolista de tracción diferencial, con paletas tipo pinball, controlado por Bluetooth desde tu celular.
- El libro se lee en orden: cada capítulo depende de lo construido en el anterior.

1 Materiales y software

EN ESTE CAPÍTULO APRENDERÁS

- Reunir todos los componentes y herramientas que vas a necesitar.
- Instalar y probar los cuatro programas que vas a usar en este libro.
- Llegar al Capítulo 2 con todo listo, sin interrupciones a mitad de camino.

Antes de tocar cualquier herramienta conviene reunir todo el material. Interrumpir el Capítulo 4 a mitad del armado porque falta un componente es la forma más común de perder el impulso de un proyecto. Este capítulo cierra ese riesgo.

1.1 Lista de materiales

La Tabla 1.1 reúne todo lo que necesitas para construir el robot completo. Ningún componente se usa antes del Capítulo 4, así que puedes seguir comprando o esperando envíos mientras avanzas con los Capítulos 2 y 3.

Tabla 1.1. Materiales necesarios para el robot futbolista.

Componente	Cant.	Rol en el robot
Arduino UNO	1	Microcontrolador, cerebro del sistema
Motor DC con reductora + rueda	2	Tracción diferencial (avance, retroceso, giro)
Módulo puente H `L298N`	1	Control de potencia y sentido de los motores DC
Servomotor (SG90 o MG90S)	2	Paletas laterales tipo pinball, para golpear el balón
Módulo Bluetooth (`HC-05` o `HC-06`)	1	Enlace inalámbrico con la app del celular
Rueda loca / caster	1	Tercer punto de apoyo del chasis
Portapilas 6×AA	1	Alimentación (9 V)
Capacitor electrolítico ≥470 µF, ≥16 V	1	Desacople del bus de 5 V cerca de los servos
Protoboard	1	Conexiones provisionales antes de fijar el cableado
Cables jumper macho-macho y macho-hembra	~20	Conexiones entre Arduino, módulos y protoboard
Filamento PLA	1 rollo	Impresión del chasis (Capítulo 3)
Pelota de tenis	1	Balón del robot

CONSEJO PRÁCTICO

Además de los componentes, necesitas herramientas básicas: un destornillador pequeño, un alicate de corte, y —muy recomendable aunque no obligatorio— un multímetro para verificar voltajes antes de conectar cada etapa nueva. Un soldador es opcional: todo el

cableado de este libro se resuelve con conectores y protoboard, sin soldadura.

NOTA

No necesitas tener impresora 3D propia. El Capítulo 3 asume que vas a usar una impresora de acceso compartido (de tu universidad, un makerspace o un servicio de impresión); lo único tuyo es el filamento y el diseño.

1.2 Software que vas a instalar

Vas a usar cuatro programas a lo largo del libro. Ninguno es gratuito por casualidad: los cuatro tienen una versión gratuita o educativa suficiente para este proyecto.

OnShape
: CAD paramétrico que corre en el navegador, sin instalación. Lo usas en el Capítulo 2 para diseñar el chasis del robot.

Ultimaker Cura
: Convierte tu diseño 3D en instrucciones que entiende la impresora (*slicing*). Lo usas en el Capítulo 3.

Arduino IDE
: Entorno donde escribes y cargas el código al Arduino UNO. Lo usas desde el Capítulo 5 en adelante.

MIT App Inventor
: Herramienta web para construir la aplicación de celular que controla el robot. La usas en el Capítulo 6.

La Figura 1.1 muestra los cuatro programas juntos.



Figura 1.1. Los cuatro programas que usarás en este libro: OnShape, Ultimaker Cura, Arduino IDE y MIT App Inventor.

MANOS A LA OBRA

Prepara tu equipo antes de continuar al Capítulo 2:

1. Crea una cuenta gratuita en `onshape.com` (solo necesitas correo).
2. Descarga e instala Ultimaker Cura desde su sitio oficial.
3. Descarga e instala el Arduino IDE desde `arduino.cc`.
4. Crea una cuenta gratuita en `appinventor.mit.edu` (con cuenta Google).
5. Abre los cuatro programas al menos una vez y confirma que inician sin errores.

ANTES DE SEGUIR, VERIFICA

- ☐ Tienes en mano (o en camino) todos los componentes de la Tabla 1.1.
- ☐ Tu cuenta de OnShape está creada y abre el editor.
- ☐ Cura y el Arduino IDE están instalados y abren sin errores.
- ☐ Tu cuenta de MIT App Inventor está creada.

RESUMEN DEL CAPÍTULO

- El robot necesita un microcontrolador, dos motores DC, dos servos, un puente H, un módulo Bluetooth y una fuente de alimentación: nada de eso se usa antes del Capítulo 4, pero conviene tenerlo listo desde ahora.
- Vas a usar cuatro programas, uno por etapa del proyecto: OnShape (diseño), Cura (impresión), Arduino IDE (programación) y MIT App Inventor (comunicación).
- No necesitas soldador ni impresora 3D propia para seguir este libro.

2 Diseño CAD en OnShape

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es el CAD paramétrico y por qué lo vas a usar.
- Moverte con soltura en la interfaz de OnShape.
- Crear un croquis con restricciones y convertirlo en una pieza sólida.
- Diseñar el chasis de tu robot y exportarlo listo para imprimir.

2.1 ¿Qué es el CAD paramétrico?

CAD (diseño asistido por computadora) es la herramienta con la que un ingeniero dibuja una pieza antes de fabricarla. La palabra clave es **paramétrico**: cada medida de tu diseño queda guardada como un número que puedes cambiar después. Si dibujas un agujero de 5 mm y luego necesitas 6 mm, no rehaces el dibujo: cambias el número.

Vas a usar **OnShape** porque corre en el navegador —sin instalación ni licencia— y porque cualquier cambio se guarda solo, en la nube.

2.2 La interfaz de OnShape

Al abrir un documento nuevo vas a ver tres zonas: el árbol de *features* a la izquierda (el historial de todo lo que has hecho), el área de trabajo en el centro, y la barra de herramientas de croquis y operaciones arriba (Figura 2.1).

NOTA

Cada pestaña en la parte inferior es un *Part Studio* o un *Assembly*. En este capítulo trabajas siempre dentro de un único Part Studio: ahí vive el chasis completo como una sola pieza.

2.3 Croquis y restricciones

Todo sólido en OnShape empieza como un croquis plano (*sketch*) dibujado sobre una cara o un plano de referencia.

1. Selecciona el plano superior (*Top*) y entra en modo croquis (*Sketch*).
2. Dibuja un rectángulo con la herramienta *Corner rectangle*.
3. Acota el ancho y el alto con la herramienta *Dimension*: escribe el valor de cada cota.



Figura 2.1. Interfaz de OnShape: árbol de features (a), área de trabajo (b) y barra de herramientas (c).

4. Centra el rectángulo en el origen: dibuja una diagonal entre dos esquinas opuestas, conviértela en línea de construcción (*Construction*) y aplica la restricción *Midpoint* entre el origen y esa diagonal. Así el rectángulo queda centrado aunque cambies las medidas después.
5. Confirma que el croquis está completamente restringido: OnShape lo dibuja en negro. Si queda algo en azul, falta una restricción o una cota.

La Figura 2.2 muestra el croquis en tres momentos: recién dibujado, al aplicar la restricción *Midpoint* y totalmente restringido.



(**a**) Rectángulo inicial



(**b**) Restricción de punto medio



(**c**) Croquis totalmente restringido

Figura 2.2. Creación de un croquis base con restricciones.

CONSEJO PRÁCTICO

Si un croquis se ve azul y no sabes qué falta, arrastra con el mouse uno de los puntos azules: lo que se mueve es lo que aún no está restringido. Si algo se ve en rojo, hay restricciones en conflicto. Cuando el croquis se complica, el *Constraint Manager* (una herramienta de diagnóstico del croquis) lista las restricciones y te ayuda a encontrar la que falta o la que sobra. Aprender a leer los colores te va a servir en cada pieza del chasis.

2.4 Operaciones básicas

Con el croquis cerrado, tres operaciones te alcanzan para la mayoría de las piezas de este proyecto:

1. **Extrusión (Extrude):** convierte el croquis plano en un sólido con espesor. Es la operación que más vas a usar.
2. **Corte (Extrude con la opción Remove):** quita material siguiendo un croquis nuevo dibujado sobre una cara existente. Así se hacen los agujeros para tornillos y los alojamientos de los servos.
3. **Redondeo (Fillet):** suaviza una arista. Además de verse mejor, una arista redondeada imprime en 3D con menos problemas que una esquina viva.

La Figura 2.3 muestra las tres operaciones aplicadas sobre la misma placa.



(**a**) Extrusión de la base



(**b**) Corte para un alojamiento



(**c**) Redondeo de aristas

Figura 2.3. Las tres operaciones básicas aplicadas sobre la placa base.

2.5 Manos a la obra: diseña tu chasis

MANOS A LA OBRA

1. Mide físicamente cada componente de la Tabla 1.1 (Capítulo 1): Arduino UNO, módulo `L298N`, motores con reductora, servos, portapilas y rueda caster.
2. Decide dónde va cada componente sobre una placa base. Deja espacio para el cableado y para que el portapilas se pueda retirar sin desarmar nada. Mantén la placa dentro del volumen de impresión de tu impresora (en muchas es de unos 220×220 mm).
3. Dibuja un boceto a mano de la vista superior, con las medidas que acabas de tomar. No lo saltes: dibujar a mano obliga a decidir la distribución antes de pelear con el mouse.
4. Recién ahora, traslada ese boceto a un croquis en OnShape y extrúyelo en la placa base.
5. Agrega los cortes para fijar cada componente (ranuras para el módulo `L298N`, alojamientos para los servos, orificios para el Arduino).

ATENCIÓN

La boca frontal de tu chasis, por donde entra el balón, debe dejar pasar al menos un tercio del diámetro de una pelota de tenis (~67 mm). Una abertura más angosta impide que el balón llegue a las paletas, sin importar qué tan bien funcione el resto del robot. Verifica esta medida en tu croquis antes de extruir.

2.6 Exportar a STL

1. En el árbol de features, selecciona la pieza terminada.
2. Haz clic derecho y elige *Export*.

3. Elige formato `STL`, resolución *Fine*, y descarga el archivo.

NOTA

Guarda el archivo con un nombre claro, por ejemplo `chasis-robot.stl`. Lo vas a necesitar abierto en Cura en el Capítulo 3.

2.7 El chasis del autor

La Figura 2.4 muestra el chasis que se usó para escribir este libro. Tómalo como referencia de proporciones y ubicación de componentes, no como una plantilla obligatoria: tu propio diseño puede —y probablemente debería— verse distinto. Sus archivos STL están en el repositorio del libro (Anexo C).

[IMAGEN PENDIENTE]

`cap02-04-chasis-autor`

Figura 2.4. Chasis de referencia del autor, con los componentes ya ubicados.

ANTES DE SEGUIR, VERIFICA

- [ ] Tu croquis base está completamente restringido (en negro, sin azul).
- [ ] El chasis tiene espacio marcado para los ocho componentes de la Tabla 1.1.
- [ ] La boca frontal deja pasar al menos un tercio del diámetro de la pelota de tenis.
- [ ] Exportaste un archivo `.stl` y lo tienes guardado.

RESUMEN DEL CAPÍTULO

- El CAD paramétrico guarda cada medida como un número editable: por eso conviene restringir bien los croquis desde el principio.
- Extrusión, corte y redondeo alcanzan para diseñar la mayoría de las piezas de este proyecto.
- El chasis se diseña a partir de mediciones reales de tus componentes, no al revés.
- La geometría de la boca frontal está condicionada por el tamaño del balón: es una

> restricción de diseño, no un detalle estético.

3 Impresión 3D

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es la manufactura aditiva y cómo imprime una impresora FDM.
- Conocer el PLA: sus temperaturas, ventajas y límites.
- Preparar tu chasis en Ultimaker Cura y convertirlo en instrucciones de impresión.
- Imprimir el chasis que diseñaste en el Capítulo 2.

3.1 Manufactura aditiva: qué es y por qué FDM

Fabricar una pieza por **manufactura sustractiva** significa partir de un bloque de material y quitarle lo que sobra —como tallar o fresar—. Fabricarla por **manufactura aditiva** significa lo contrario: construirla agregando material capa por capa, desde cero, hasta completar la forma.

La tecnología que vas a usar es **FDM** (Modelado por Deposición Fundida): un filamento sólido se funde en una boquilla caliente y se deposita capa sobre capa, siguiendo exactamente el camino que definiste al diseñar el chasis en OnShape. En la Figura 3.1 se ven las tres partes que importan: la boquilla, la cama caliente y los ejes de movimiento.

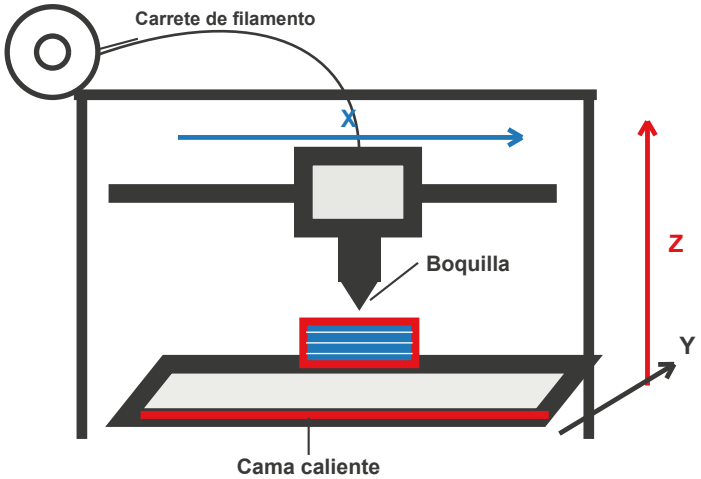


Figura 3.1. Anatomía de una impresora FDM: boquilla, cama caliente y ejes de movimiento.

3.2 El material: PLA

Vas a imprimir en **PLA** (ácido poliláctico), el material más usado para empezar en impresión 3D: es fácil de imprimir, no necesita cama caliente a temperaturas extremas y no desprende gases fuertes.

Tabla 3.1. PLA comparado con otros filamentos comunes.

Material	Boquilla	Cama	Para este proyecto
PLA	190–220 °C	50–60 °C	Recomendado: fácil, rígido, suficiente para el chasis
PETG	230–250 °C	70–80 °C	Más resistente al impacto, más difícil de imprimir bien
ABS	220–250 °C	90–110 °C	Requiere recinto cerrado; se deforma (*warping*) sin él

NOTA

El PLA es más frágil que el PETG o el ABS a temperatura ambiente, pero para un chasis que no queda expuesto al sol ni a temperaturas altas, esa diferencia no importa. Empieza con PLA; puedes reimprimir en otro material más adelante si una pieza en particular lo necesita.

3.3 Diseño para impresión 3D

Antes de laminar el chasis del Capítulo 2, revisa estos cuatro puntos: una impresora FDM no fabrica cualquier geometría igual de bien.

- **Voladizos:** una superficie que sobresale en el aire sin apoyo necesita soporte, o se cae mientras se imprime. Si puedes evitarlo reorientando la pieza, hazlo.
- **Orientación:** la pieza es más fuerte a lo largo de las capas que entre ellas. Orienta el chasis para que las fuerzas del uso normal —el peso de los motores, el impacto del balón contra las paletas— no tiendan a separar capas.
- **Anisotropía:** por lo anterior, una pieza impresa en 3D nunca es igual de resistente en todas direcciones. No es un defecto de tu impresión: es una propiedad del proceso.
- **Agujeros y tornillos:** un agujero impreso sale ligeramente más angosto que su medida nominal. Diseña los agujeros para tornillos con 0.2–0.3 mm de holgura extra.

3.4 Laminando en Cura

Laminar (*slicing*) es convertir tu modelo 3D en instrucciones capa por capa que la impresora entiende: el código G.

1. Abre Cura, elige el perfil de tu impresora e importa el archivo `chasis-robot.stl` que exportaste en el Capítulo 2. Si el modelo aparece con rayas grises, se sale del volumen de impresión: vuelve a OnShape y redúcelo.

2. Selecciona el perfil de material PLA y ajusta la altura de capa (0.2 mm es un buen punto de partida: buen acabado sin imprimir demasiado lento).
3. Define el relleno (20 % alcanza para el chasis) y activa soportes solo en los voladizos que identificaste en la sección anterior.
4. Verifica la adherencia a la cama: un borde (*brim*) ayuda si la pieza tiene poca superficie de contacto con la base.
5. Exporta el código G a la tarjeta SD o envíalo directo a la impresora.

La Figura 3.2 muestra el modelo importado, los ajustes y la vista previa por capas.



(**a**) Modelo importado (**b**) Ajustes de impresión (**c**) Vista previa por capas

Figura 3.2. Preparación del chasis en Ultimaker Cura.

CONSEJO PRÁCTICO

La vista previa por capas de Cura simula la impresión completa antes de gastar filamento. Revísala siempre: si ves soportes flotando en el aire o una base de contacto muy pequeña, corrige la orientación antes de imprimir, no después.

3.5 Manos a la obra: imprime tu chasis

MANOS A LA OBRA

1. Carga el código G de tu chasis en la impresora.
2. Nivela la cama si la impresora lo requiere, siguiendo su manual.
3. Inicia la impresión y supervisa las primeras capas: son las que definen si la pieza se pega bien a la base.
4. Cuando termine, retira la pieza con cuidado y quita los soportes con un alicate o una espátula.

3.6 Fallas típicas

ANTES DE SEGUIR, VERIFICA

- ☐ Identificaste los voladizos de tu chasis y decidiste su orientación.
- ☐ El chasis cabe en el volumen de impresión: en Cura no aparece con rayas grises.
- ☐ Laminaste el modelo en Cura con altura de capa 0.2 mm y relleno 20 %.
- ☐ Revisaste la vista previa por capas antes de imprimir.

Tabla 3.2. Diagnóstico rápido de fallas comunes en impresión FDM.

Síntoma	Causa probable	Solución
Las esquinas se levantan de la cama (*warping*)	Mala adherencia o enfriamiento desigual	Usa *brim*, limpia la cama, evita corrientes de aire
Hilos finos entre piezas (*stringing*)	Temperatura muy alta o falta de retracción	Baja 5–10 °C la boquilla, activa retracción
La pieza se despega a mitad de impresión	Cama sucia, sin nivelar, o primera capa muy rápida	Limpia con alcohol isopropílico, nivela, reduce velocidad de la primera capa

☐ Tu chasis está impreso, sin soportes, y encaja con las medidas que tomaste en el Capítulo 2.

RESUMEN DEL CAPÍTULO

- La manufactura aditiva construye piezas capa por capa; la FDM lo hace fundiendo filamento.
- El PLA es el material correcto para empezar: fácil de imprimir y suficiente para este chasis.
- Orientación, voladizos y anisotropía son decisiones de diseño, no detalles de la impresora.
- La mayoría de las fallas de impresión se resuelven en la primera capa o en la temperatura, no cambiando de máquina.

4 Electrónica

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué hace cada componente eléctrico y electrónico del robot.
- Saber por qué un motor DC nunca se conecta directo al Arduino.
- Calcular y armar la alimentación del robot sin quemar ningún componente.
- Cablear y montar todos los componentes sobre tu chasis, en etapas que puedes verificar una por una.

Este es el capítulo donde más fácil es dañar un componente. No es motivo para tener miedo, sino para ir por etapas: cada sección de armado más adelante trae su propia verificación, y no vas a energizar nada que no puedas revisar antes.

4.1 Conceptos base

Tres cantidades explican casi todo lo que vas a hacer en este capítulo. El **voltaje** es la presión eléctrica disponible, en voltios (V). La **corriente** es cuánta electricidad realmente fluye, en amperios (A) — un motor detenido a la fuerza pide más corriente, no más voltaje. La **resistencia**, en ohmios (Ω), es lo que se opone a ese flujo. Y **GND** (tierra o masa) es el punto de referencia común: todo voltaje se mide *respecto a* GND, así que todos los GND del robot deben estar conectados entre sí.

4.2 Descripción de componentes

Tabla 4.1. Los cuatro componentes centrales de este capítulo.

Componente	Qué hace	Dato clave
Motor DC + reductora	Convierte electricidad en giro; la reductora baja las RPM y multiplica el torque	Pide cientos de mA al arrancar
Servomotor (SG90/MG90S)	Gira hasta un ángulo específico según la señal que recibe; no da vueltas completas	Señal PWM a 50 Hz, pulso de 1–2 ms
Módulo `L298N`	Puente H: controla sentido y velocidad de motores DC que piden más corriente de la que entrega el Arduino	Hasta ~2 A por canal; cae ~2 V
Arduino UNO	Microcontrolador: ejecuta el programa y genera las señales de control	Cada pin entrega máx. 40 mA

4.3 Por qué necesitas un puente H

ATENCIÓN

Nunca conectes un motor DC directamente a un pin del Arduino. El pin entrega como máximo 40 mA y el motor demanda cientos de miliamperios en el arranque. Vas a quemar el pin, y en el peor caso, el microcontrolador completo.

El módulo `L298N` resuelve esto: el Arduino le manda una señal de control de apenas unos miliamperios, y el `L298N` —con su propia alimentación— mueve la corriente real que el motor necesita. Además, invierte la polaridad del motor según la señal, lo que te permite elegir el sentido de giro sin tocar cables.

La velocidad se controla con **PWM** (Modulación por Ancho de Pulso): en vez de entregar un voltaje constante, el pin enciende y apaga muchas veces por segundo. Cuanto más tiempo pasa encendido en cada ciclo, más rápido gira el motor. `ENA` y `ENB` son los pines del `L298N` que reciben esa señal PWM, uno por motor.

Tabla 4.2. Conexión entre el Arduino UNO y el módulo L298N.

Arduino	L298N	Función
`D10`	`ENA`	Velocidad del motor izquierdo (PWM)
`D8`	`IN1`	Sentido del motor izquierdo
`D9`	`IN2`	Sentido del motor izquierdo
`D5`	`ENB`	Velocidad del motor derecho (PWM)
`D6`	`IN3`	Sentido del motor derecho
`D7`	`IN4`	Sentido del motor derecho
`GND`	`GND`	Masa común, obligatoria

4.4 Servomotores y ancho de pulso

Un servomotor también se controla por ancho de pulso, pero con otro significado: el pulso —repetido 50 veces por segundo— le dice al servo *a qué ángulo* moverse, no qué tan rápido girar. Un pulso de 1 ms lo manda a un extremo, uno de 2 ms al otro, y 1.5 ms al centro.

Tus dos servos van conectados a `D3` y `D11` (paleta izquierda y derecha), ambos pines compatibles con PWM.

ATENCIÓN

Cada paleta necesita un **tope mecánico** y un **rango angular acotado por software**. Un servo SG90 fuerza su engranaje plástico —y lo rompe— si el brazo choca contra el chasis antes de terminar su recorrido. En el Capítulo 5 vas a limitar el ángulo de cada servo en el código; por ahora, al montar la paleta, verifica a mano que se mueve libre en todo su rango antes de energizarla.

4.5 Alimentación

Tu robot se alimenta con **6 pilas AA en serie**, que entregan ~9 V. Esa energía tiene que repartirse entre motores, servos y el propio Arduino sin que ninguna etapa le quite margen a las demás.

NOTA

El `L298N` pierde ~2 V al controlar los motores: con 9 V de entrada, los motores reciben ~7 V, no los 9 V nominales. Es normal, y conviene saberlo para no esperar más torque o velocidad de la que el sistema realmente entrega.

El módulo `L298N` trae un jumper que, si lo dejas puesto, habilita un regulador interno que entrega 5 V a partir de la entrada de 9 V —útil para alimentar el Arduino sin una fuente aparte.

ATENCIÓN

En este montaje, el Arduino y los dos servos comparten la salida de 5 V del `L298N`. El regulador de algunos módulos puede no entregar corriente suficiente para las dos paletas bajo carga: una caída de tensión puede reiniciar el Arduino. Instala un capacitor electrolítico de al menos 470 µF (16 V o más) entre 5 V y GND, cerca de los servos y con la polaridad correcta. **El capacitor no aumenta la capacidad del regulador.** Mide la tensión mientras se mueven ambas paletas; si cae o el Arduino se reinicia, detén la prueba y usa una fuente regulada de 5 V con corriente suficiente para los servos, manteniendo GND común.

ATENCIÓN

GND común obligatorio. Las pilas, el `L298N`, el Arduino y los servos deben compartir la misma referencia de tierra. Sin GND común, las señales de control pierden su punto de referencia y el robot se comporta de forma errática o no responde.

CONSEJO PRÁCTICO

Las pilas AA alcalinas pierden voltaje rápido bajo el consumo de dos motores y dos servos, y ese voltaje caído se nota como pérdida de fuerza a medida que avanza la sesión de pruebas. Si notas que el robot rinde menos después de media hora, no es tu cableado: son las pilas. Las AA recargables NiMH entregan más corriente sostenida y son la mejora más simple que puedes hacerle al robot.

4.6 Manos a la obra: cableado y ensamble

Arma el robot en las siguientes etapas, en orden. Cada una trae su propia verificación: no avances a la siguiente sin pasarla. La Figura 4.1 muestra el esquema completo del cableado; consúltala en cada etapa.

MANOS A LA OBRA

1. **Portapilas al L298N.** Conecta las 6 pilas AA a la entrada de alimentación del módulo `L298N`. *Prueba:* el LED de encendido del módulo se ilumina.
2. **Salida de 5 V del L298N.** Confirma que el jumper de 5 V está puesto. *Prueba:* con el

multímetro, mide ~5 V entre el pin de 5 V del módulo y GND.

3. **Arduino alimentado.** Conecta esa salida regulada de 5 V al pin `5V` del Arduino y su GND a la masa común. *Prueba:* el Arduino enciende y su LED integrado no parpadea de forma errática.
4. **Motores al L298N.** Conecta cada motor a sus salidas `OUT1`/`OUT2` y `OUT3`/`OUT4`. *Prueba:* con el multímetro en modo continuidad, confirma que no hay corto entre las dos salidas de un mismo motor.
5. **Señales de control al Arduino.** Cablea `ENA`, `IN1`–`IN4` y `ENB` según la Tabla 4.2. *Prueba:* revisa cable por cable contra la tabla; un pin cruzado aquí es el error más común de todo el capítulo.
6. **Servos al bus de 5 V compartido.** Lleva la salida de 5 V y GND del `L298N` a la protoboard, conecta el capacitor con su polo positivo a 5 V y el negativo a GND, y después conecta los servos. Sus señales van a `D3` y `D11`. *Prueba:* verifica cada cable (rojo = 5 V, café o negro = GND, naranjo o amarillo = señal); luego mide 5 V mientras ambas paletas se mueven.
7. **Verificación de GND común.** Con todo cableado y *sin* pilas puestas, mide continuidad entre el GND del Arduino, el GND del `L298N` y el GND de los servos. *Prueba:* el multímetro debe marcar continuidad (~0 Ω) entre los tres puntos.
8. **Montaje sobre el chasis.** Fija cada componente en su lugar en el chasis impreso en el Capítulo 3, con el cableado ordenado y sin tensión en los conectores.

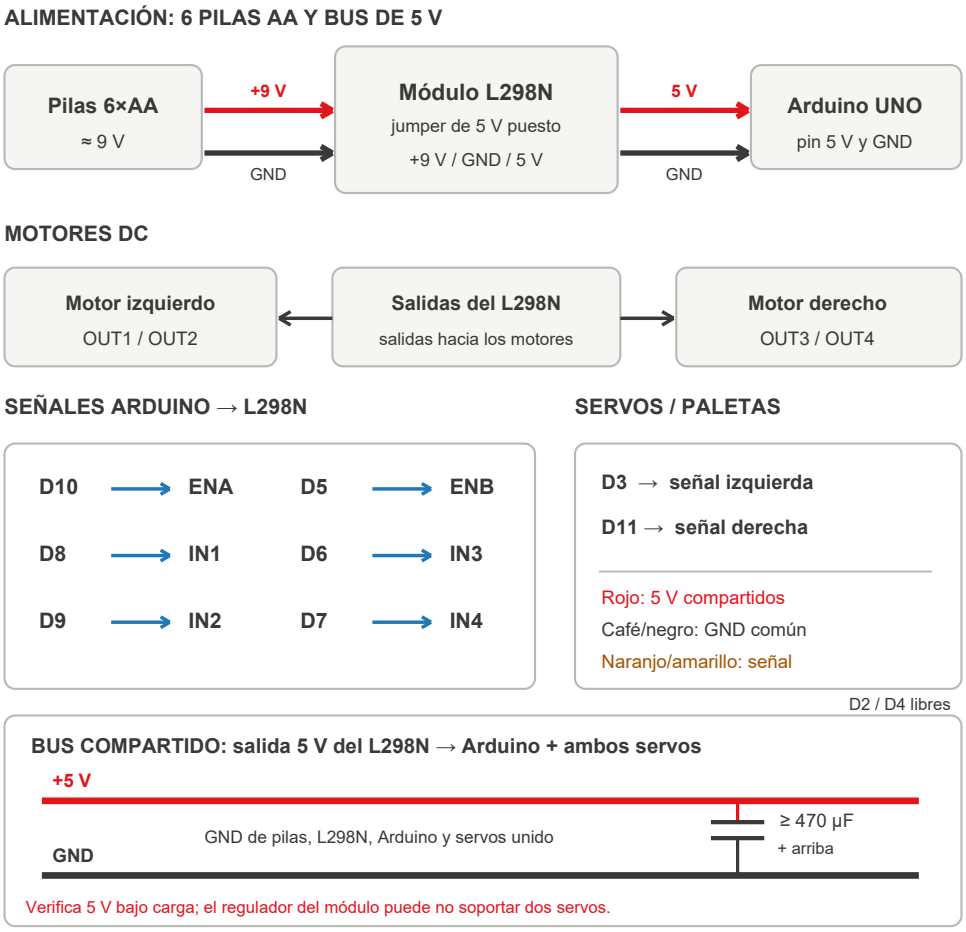


Figura 4.1. Esquema de conexión completo: pilas, L298N, Arduino, motores y servos.

ANTES DE SEGUIR, VERIFICA

- [ ] El LED del módulo `L298N` enciende al conectar las pilas.
- [ ] La salida de 5 V del módulo mide ~5 V con el multímetro.
- [ ] El Arduino enciende sin parpadeos ni reinicios.
- [ ] Cada pin de la Tabla 4.2 está cableado en el lugar correcto.
- [ ] Los servos comparten el bus de 5 V, con el capacitor de desacople instalado y la tensión verificada bajo carga.
- [ ] Hay continuidad de GND entre Arduino, L298N y servos.
- [ ] Todos los componentes están fijos sobre el chasis.

RESUMEN DEL CAPÍTULO

- Un motor DC nunca se conecta directo al Arduino: el L298N existe para manejar esa corriente.
- PWM controla velocidad en los motores y ángulo en los servos, con el mismo principio de pulsos y distinto significado.
- El bus de 5 V del L298N alimenta al Arduino y a los servos en este montaje; hay que verificar su tensión bajo carga. El capacitor reduce caídas breves, pero no reemplaza una fuente con corriente suficiente.
- GND común no es opcional: sin él, el robot se comporta de forma errática aunque todo lo demás esté bien cableado.
- El armado en etapas verificables evita que un error de cableado aparezca recién cuando todo está montado y es difícil de revisar.

5 Programación

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es programar y por qué Arduino usa C++.
- Reconocer la estructura de un programa Arduino: `setup()` y `loop()`.
- Escribir funciones que muevan un motor, luego dos, y finalmente ambos servos.
- Cargar una rutina de prueba completa y verificar que todo el robot responde.

5.1 ¿Qué es programar?

Programar es escribir instrucciones que una máquina ejecuta en orden, una tras otra. Un lenguaje de programación es *de bajo nivel* cuando esas instrucciones están cerca de lo que el procesador entiende directamente, y *de alto nivel* cuando están más cerca de cómo piensa una persona.

Arduino se programa en **C++**, un lenguaje de nivel medio: te deja controlar cada pin directamente, pero con una sintaxis más cómoda que el ensamblador. La razón práctica es que un Arduino UNO tiene apenas 2 KB de memoria para variables: C++ te da ese control fino sin desperdiciarla.

5.2 Estructura de un programa Arduino

Todo programa de Arduino tiene dos funciones obligatorias:

- `setup()` se ejecuta una sola vez, al encender o reiniciar. Aquí configuras los pines.
- `loop()` se repite sin parar mientras el Arduino tenga corriente. Aquí vive el comportamiento del robot.

El Código 5.1 usa las tres instrucciones más básicas: `pinMode` declara si un pin es entrada o salida, `digitalWrite` lo pone en `HIGH` o `LOW`, y una variable (`const int LED`) le da nombre a un número de pin para no repetirlo por todo el código.

```
// Enciende y apaga el LED integrado del Arduino cada segundo.
const int LED = LED_BUILTIN;

void setup() {
  pinMode(LED, OUTPUT);
}

void loop() {
  digitalWrite(LED, HIGH);
  delay(1000);
  digitalWrite(LED, LOW);
  delay(1000);
```

```
}
```

Código 5.1. Parpadeo del LED integrado del Arduino.

> **NOTA**
>
> `analogWrite` funciona distinto de `digitalWrite`: en vez de `HIGH`/`LOW`, recibe un número de 0 a 255 y genera una señal PWM. Es la instrucción que vas a usar para controlar la velocidad de los motores.

5.3 De un motor a dos: funciones y tracción diferencial

Una **función** le pone nombre a un bloque de instrucciones que vas a repetir. Sin funciones, mover dos motores en cuatro combinaciones de sentido —avanzar, retroceder, girar a cada lado— te obliga a repetir el mismo patrón de `digitalWrite` una y otra vez, con más oportunidades de equivocarte en algún cable.

El Código 5.2 define una función `moverMotor()` que recibe qué pines usar, a qué velocidad y en qué sentido, y encima construye `avanzar()`, `retroceder()`, `girarIzquierda()` y `girarDerecha()` llamando a esa función una vez por motor.

```
// Control de los dos motores DC vía el módulo L298N (ver Tabla 4.2, Capítulo 4).
const int ENA = 10, IN1 = 8, IN2 = 9;  // Motor izquierdo
const int ENB = 5,  IN3 = 6, IN4 = 7;  // Motor derecho

// Mueve un motor: adelante o atrás, a la velocidad indicada (0-255).
void moverMotor(int in1, int in2, int en, int velocidad, bool adelante) {
  digitalWrite(in1, adelante ? HIGH : LOW);
  digitalWrite(in2, adelante ? LOW : HIGH);
  analogWrite(en, velocidad);
}

void avanzar(int velocidad) {
  moverMotor(IN1, IN2, ENA, velocidad, true);
  moverMotor(IN3, IN4, ENB, velocidad, true);
}

void retroceder(int velocidad) {
  moverMotor(IN1, IN2, ENA, velocidad, false);
  moverMotor(IN3, IN4, ENB, velocidad, false);
}

void girarIzquierda(int velocidad) {
  moverMotor(IN1, IN2, ENA, velocidad, false);
  moverMotor(IN3, IN4, ENB, velocidad, true);
}

void girarDerecha(int velocidad) {
  moverMotor(IN1, IN2, ENA, velocidad, true);
  moverMotor(IN3, IN4, ENB, velocidad, false);
}

void detener() {
  analogWrite(ENA, 0);
  analogWrite(ENB, 0);
}
```

```
36
37 void setup() {
38   pinMode(IN1, OUTPUT);
39   pinMode(IN2, OUTPUT);
40   pinMode(ENA, OUTPUT);
41   pinMode(IN3, OUTPUT);
42   pinMode(IN4, OUTPUT);
43   pinMode(ENB, OUTPUT);
44 }
45
46 void loop() {
47   avanzar(150);
48   delay(1000);
49   detener();
50   delay(500);
51 }
```

Código 5.2. Funciones de movimiento sobre el módulo L298N.

CONSEJO PRÁCTICO

Si tu robot avanza cuando debería retroceder, no cambies el código: lo más probable es que un motor esté cableado al revés. Intercambia sus dos cables en la bornera del L298N antes de tocar una sola línea de `moverMotor()`.

5.4 Servos y la función de pateo

La librería `Servo.h` simplifica el control por ancho de pulso que viste en el Capítulo 4: en vez de calcular pulsos, llamas a `servo.write(ángulo)` con un número de 0 a 180.

El Código 5.3 declara los dos servos, los deja en su ángulo de reposo, y define `patear()`: mueve la paleta al ángulo de pateo y la regresa al reposo. `ANGULO_PATEO` está fijado en 40°, bien lejos del límite mecánico del servo, siguiendo la advertencia del Capítulo 4 sobre no forzar el engranaje.

```
1  // Control de las dos paletas (servos) con rango angular acotado.
2  // El límite entre reposo y pateo evita forzar el engranaje (ver Capítulo 4).
3  #include <Servo.h>
4
5  Servo servoIzquierdo, servoDerecho;
6  const int PIN_SERVO_IZQ = 3, PIN_SERVO_DER = 11;
7  const int ANGULO_REPOSO = 90;
8  const int ANGULO_PATEO = 40;
9
10 void patear(Servo &servo) {
11   servo.write(ANGULO_PATEO);
12   delay(200);
13   servo.write(ANGULO_REPOSO);
14 }
15
16 void setup() {
17   servoIzquierdo.attach(PIN_SERVO_IZQ);
18   servoDerecho.attach(PIN_SERVO_DER);
19   servoIzquierdo.write(ANGULO_REPOSO);
20   servoDerecho.write(ANGULO_REPOSO);
21 }
22
23 void loop() {
```

```
  patear(servoIzquierdo);
  delay(1000);
  patear(servoDerecho);
  delay(1000);
}
```

Código 5.3. Control de las paletas con rango angular acotado.

5.5 Manos a la obra: tu rutina de prueba

MANOS A LA OBRA

1. Descarga del repositorio del libro (Anexo C) el archivo `codigo/rutina-prueba/rutina-prueba.ino`, que integra las funciones de las dos secciones anteriores en un solo programa, y ábrelo en el Arduino IDE.
2. Conecta el Arduino a tu computadora y súbelo con el botón de carga.
3. Con el robot sobre una superficie donde las ruedas giren libres, enciende la alimentación de motores y servos.
4. Observa el ciclo completo: avanza, retrocede, gira a la izquierda, gira a la derecha, patea con el servo izquierdo y patea con el derecho.

ATENCIÓN

Si alguna función no responde —un motor no gira, un servo no se mueve— **no insistas con el código**. Vuelve al Capítulo 4 y revisa esa etapa específica del armado: en la enorme mayoría de los casos, un motor o servo mudo es un cable en el pin equivocado o una alimentación mal conectada, no un error de programación.

5.6 Depuración con el Monitor Serie

La rutina de prueba llama a `Serial.begin(9600)` en `setup()`, y a `Serial.println()` antes de cada movimiento. Abre el Monitor Serie del Arduino IDE, con el Arduino ya cargado y conectado, para ver en texto qué función se está ejecutando en cada momento.

CONSEJO PRÁCTICO

El Monitor Serie es la herramienta más simple para saber si tu programa llegó a una parte del código: si el mensaje «Girar izquierda» nunca aparece, el problema está antes de esa línea, no después.

ANTES DE SEGUIR, VERIFICA

- ☐ El LED integrado del Arduino parpadea al cargar el Código 5.1.
- ☐ El robot avanza y retrocede en línea recta al ejecutar `avanzar()` y `retroceder()`.
- ☐ El robot gira hacia cada lado al ejecutar `girarIzquierda()` y `girarDerecha()`.

- [ ] Ambos servos patean y vuelven a su posición de reposo sin forzar el engranaje.
- [ ] El Monitor Serie muestra el nombre de cada movimiento mientras el robot lo ejecuta.

RESUMEN DEL CAPÍTULO

- Todo programa Arduino tiene `setup()` (una vez) y `loop()` (siempre).
- Las funciones evitan repetir código y hacen visibles los errores de cableado: si `avanzar()` falla, sabes exactamente qué revisar.
- La tracción diferencial se logra combinando el sentido de giro de los dos motores, no con hardware adicional.
- El rango angular de los servos se acota en el código para proteger el engranaje físico.
- El Monitor Serie es tu primera herramienta de diagnóstico, antes de sospechar del cableado.

6 Comunicación

EN ESTE CAPÍTULO APRENDERÁS

- Entender qué es un protocolo de comunicación y dónde encaja Bluetooth.
- Conectar el módulo HC-06 al Arduino sin dañarlo.
- Definir el protocolo de comandos de tu robot y programarlo para recibirlos.
- Construir en MIT App Inventor una aplicación que controle tu robot desde el celular.

6.1 ¿Qué es un protocolo, y por qué Bluetooth?

Un **protocolo** es un acuerdo: dos dispositivos deciden de antemano qué significa cada dato que se envían, para poder entenderse. Un cable USB y una red WiFi son ambos «alámbrico» y «inalámbrico», pero por debajo usan protocolos distintos: `UART`, `I2C` y `SPI` para conectar chips dentro de un mismo circuito; WiFi y **Bluetooth** para comunicar dispositivos separados sin cables.

Vas a usar Bluetooth porque todo celular moderno lo trae, y porque el módulo que necesitas para hablar con él desde un Arduino —el HC-06— es barato y simple de programar.

6.2 El módulo HC-06 y SoftwareSerial

El **HC-06** es un módulo Bluetooth que convierte lo que envías por comunicación serial en una señal Bluetooth, y viceversa. Desde el punto de vista del Arduino, hablar con el HC-06 es igual de simple que hablar con el Monitor Serie del Capítulo 5.

ATENCIÓN

Si conectas el HC-06 a los pines seriales de hardware del Arduino (`RX` en `D0`, `TX` en `D1`), tienes que desconectarlo cada vez que subes un programa nuevo: esos mismos pines los usa el puerto USB para cargar el código, y ambos usos chocan.

Para evitar ese problema por completo, vas a usar la librería `SoftwareSerial`: crea un puerto serial «de software» en dos pines digitales cualquiera, dejando los pines de hardware libres para programar y para el Monitor Serie al mismo tiempo.

ATENCIÓN

El HC-06 espera **3.3 V** en su entrada `RXD`, pero el Arduino entrega **5 V** en sus pines digitales. Conectarlo directo arriesga el módulo. Arma un divisor de tensión con dos resistencias entre el pin `D4` del Arduino y `RXD`: 1 kΩ en serie hacia `RXD`, y 2 kΩ desde `RXD` a GND. Ese par entrega ~3.3 V al módulo. El sentido `TXD`→`D2` no necesita divisor: el HC-06 ya entrega una

Tabla 6.1. Conexión entre el Arduino UNO y el módulo HC-06.

Arduino	HC-06	Función
D2	TXD	El HC-06 transmite, el Arduino recibe
D4	RXD	El Arduino transmite, a través del divisor de tensión
5V	VCC	Alimentación del módulo
GND	GND	Masa común, obligatoria

señal que el Arduino reconoce sin problema.

6.3 El protocolo de comandos del robot

Vas a definir el protocolo más simple posible: un carácter, un comando.

Tabla 6.2. Protocolo de comandos del robot.

Carácter	Acción
F	Avanzar
B	Retroceder
L	Girar a la izquierda
R	Girar a la derecha
S	Detenerse
K	Patear (ambas paletas)

El Código 6.1 retoma las funciones de movimiento del Capítulo 5 y agrega `ejecutarComando()`: lee un carácter desde el `SoftwareSerial` y llama a la función correspondiente. En `loop()`, el Arduino solo pregunta si llegó un comando nuevo —no hace falta nada más.

```
// Control del robot por Bluetooth (módulo HC-06 vía SoftwareSerial).
// Protocolo de un carácter por comando: F, B, L, R, S, K (ver Tabla 6.2).
#include <SoftwareSerial.h>
#include <Servo.h>

SoftwareSerial bluetooth(2, 4);  // RX, TX (TX va al HC-06 a través del divisor)

// Motores (módulo L298N, ver Tabla 4.2, Capítulo 4)
const int ENA = 10, IN1 = 8, IN2 = 9;  // Motor izquierdo
const int ENB = 5,  IN3 = 6, IN4 = 7;  // Motor derecho
const int VELOCIDAD = 180;

// Servos: paletas tipo pinball (Capítulo 4)
Servo servoIzquierdo, servoDerecho;
const int PIN_SERVO_IZQ = 3, PIN_SERVO_DER = 11;
const int ANGULO_REPOSO = 90;
const int ANGULO_PATEO = 40;

void moverMotor(int in1, int in2, int en, int velocidad, bool adelante) {
  digitalWrite(in1, adelante ? HIGH : LOW);
  digitalWrite(in2, adelante ? LOW : HIGH);
  analogWrite(en, velocidad);
}
```

```
24
25 void avanzar()        { moverMotor(IN1, IN2, ENA, VELOCIDAD, true);
26                        moverMotor(IN3, IN4, ENB, VELOCIDAD, true); }
27 void retroceder()     { moverMotor(IN1, IN2, ENA, VELOCIDAD, false);
28                        moverMotor(IN3, IN4, ENB, VELOCIDAD, false); }
29 void girarIzquierda() { moverMotor(IN1, IN2, ENA, VELOCIDAD, false);
30                        moverMotor(IN3, IN4, ENB, VELOCIDAD, true); }
31 void girarDerecha()   { moverMotor(IN1, IN2, ENA, VELOCIDAD, true);
32                        moverMotor(IN3, IN4, ENB, VELOCIDAD, false); }
33 void detener()        { analogWrite(ENA, 0); analogWrite(ENB, 0); }
34
35 void patear() {
36   servoIzquierdo.write(ANGULO_PATEO);
37   servoDerecho.write(ANGULO_PATEO);
38   delay(200);
39   servoIzquierdo.write(ANGULO_REPOSO);
40   servoDerecho.write(ANGULO_REPOSO);
41 }
42
43 void ejecutarComando(char comando) {
44   switch (comando) {
45     case 'F': avanzar(); break;
46     case 'B': retroceder(); break;
47     case 'L': girarIzquierda(); break;
48     case 'R': girarDerecha(); break;
49     case 'S': detener(); break;
50     case 'K': patear(); break;
51   }
52 }
53
54 void setup() {
55   pinMode(IN1, OUTPUT);
56   pinMode(IN2, OUTPUT);
57   pinMode(ENA, OUTPUT);
58   pinMode(IN3, OUTPUT);
59   pinMode(IN4, OUTPUT);
60   pinMode(ENB, OUTPUT);
61
62   servoIzquierdo.attach(PIN_SERVO_IZQ);
63   servoDerecho.attach(PIN_SERVO_DER);
64   servoIzquierdo.write(ANGULO_REPOSO);
65   servoDerecho.write(ANGULO_REPOSO);
66
67   bluetooth.begin(9600);
68 }
69
70 void loop() {
71   if (bluetooth.available()) {
72     ejecutarComando(bluetooth.read());
73   }
74 }
```

Código 6.1. Recepción de comandos por Bluetooth y control del robot.

6.4 MIT App Inventor: construye la app

MIT App Inventor tiene dos vistas: el **Diseñador**, donde arrastras botones y otros elementos visuales, y **Bloques**, donde defines qué hace cada uno.

1. En el Diseñador, agrega un componente `BluetoothClient` (no visible), cinco botones —adelante, atrás, izquierda, derecha y patear— y un selector de lista (`ListPicker`) con el texto «Conectar».
2. En Bloques, programa la conexión: antes de abrirse (`BeforePicking`), el selector carga en `Elements` los dispositivos emparejados (`BluetoothClient1.AddressesAndNames`); al elegir uno (`AfterPicking`), llama a `BluetoothClient1.Connect` con la selección.
3. En cada botón de dirección usa dos eventos: `TouchDown` (al presionar) y `TouchUp` (al soltar). En `TouchDown` llama a `BluetoothClient1.SendText` con la letra de esa dirección, según la Tabla 6.2; en `TouchUp` envía `S`. Sin este segundo bloque el robot no se detiene: el programa del Arduino solo frena al recibir `S`.
4. En el botón de patear basta el evento `Click`, que envía `K`.

La Figura 6.1 muestra el Diseñador, los Bloques y la app ya corriendo en el celular.

[IMAGEN PENDIENTE]
`cap06-01a-disenador`

(**a**) Diseñador: botones

[IMAGEN PENDIENTE]
`cap06-01b-bloques`

(**b**) Bloques: enviar comando

[IMAGEN PENDIENTE]
`cap06-01c-app-celular`

(**c**) App corriendo en el celular

Figura 6.1. Construcción de la app de control en MIT App Inventor.

CONSEJO PRÁCTICO

Prueba tu app con el emulador o con la app «MIT AI2 Companion» antes de exportarla como `.apk`. Así detectas errores de bloques sin esperar a tener el robot encendido al lado.

6.5 Manos a la obra: empareja y controla tu robot

MANOS A LA OBRA

1. Enciende el robot y busca el HC-06 en el Bluetooth de tu celular. El código de emparejamiento por defecto suele ser `1234` o `0000`.
2. Instala tu app (exportada como `.apk`, o desde el Companion) en el celular ya emparejado.
3. Abre la app, conecta al HC-06, y prueba cada botón: adelante, atrás, izquierda, derecha y patear. Mantén presionado un botón de dirección y suéltalo: el robot debe moverse mientras lo presionas y detenerse al soltarlo.
4. Si un botón no responde, revisa primero el carácter que envía en Bloques contra la Tabla 6.2: es el error más común en esta etapa. Si el robot no se detiene al soltar, falta el bloque `TouchUp` que envía `S`.

ANTES DE SEGUIR, VERIFICA

- ☐ El celular encuentra y empareja el HC-06.
- ☐ La app se conecta al módulo sin errores.

- [ ] Los cuatro botones de dirección mueven el robot mientras los presionas y lo detienen al soltarlos.
- [ ] El botón de patear activa ambas paletas.

RESUMEN DEL CAPÍTULO

- Un protocolo es un acuerdo sobre qué significa cada dato: el tuyo tiene seis comandos, cada uno una sola letra.
- `SoftwareSerial` evita el conflicto entre el HC-06 y el puerto USB de programación.
- El HC-06 necesita un divisor de tensión en su entrada: nunca recibe directo los 5 V del Arduino.
- MIT App Inventor convierte cada botón en un mensaje Bluetooth de un carácter, que tu Arduino ya sabe interpretar.

7 Epílogo

Felicidades. Si llegaste hasta aquí con el robot respondiendo a tu celular, no armaste solo un vehículo con motores: pasaste por las cuatro capas que el Capítulo 0 prometía —mecánica, eléctrica, electrónica e informática— y las hiciste convivir en una sola máquina.

ANTES DE SEGUIR, VERIFICA

- [ ] El chasis diseñado en OnShape está impreso y todos los componentes encajan en él.
- [ ] Los motores, el módulo L298N, el Arduino y los servos están cableados y alimentados sin GND flotante.
- [ ] El código de la rutina de prueba corrió sin errores en los seis movimientos.
- [ ] Tu app de MIT App Inventor controla el robot por Bluetooth: avanza, retrocede, gira y patea.

7.1 Reglas para jugar

No necesitas más que una cancha improvisada y otro robot igual al tuyo. Marca un rectángulo de unos 2×1 m con cinta en el suelo, con un arco pequeño en cada extremo. Dos reglas alcanzan: gana quien mete el balón en el arco contrario, y ningún robot puede tocar al otro directamente —solo al balón.

7.2 Lo que aprendiste

Lo que realmente construiste es transferible: diseñar una pieza para que cumpla una función, entender por qué un motor necesita un puente H antes de recibir corriente, escribir funciones en vez de repetir código, y definir un protocolo simple para que dos dispositivos se entiendan. Esas cuatro habilidades sirven para el próximo robot, no solo para este.

7.3 Hacia dónde seguir

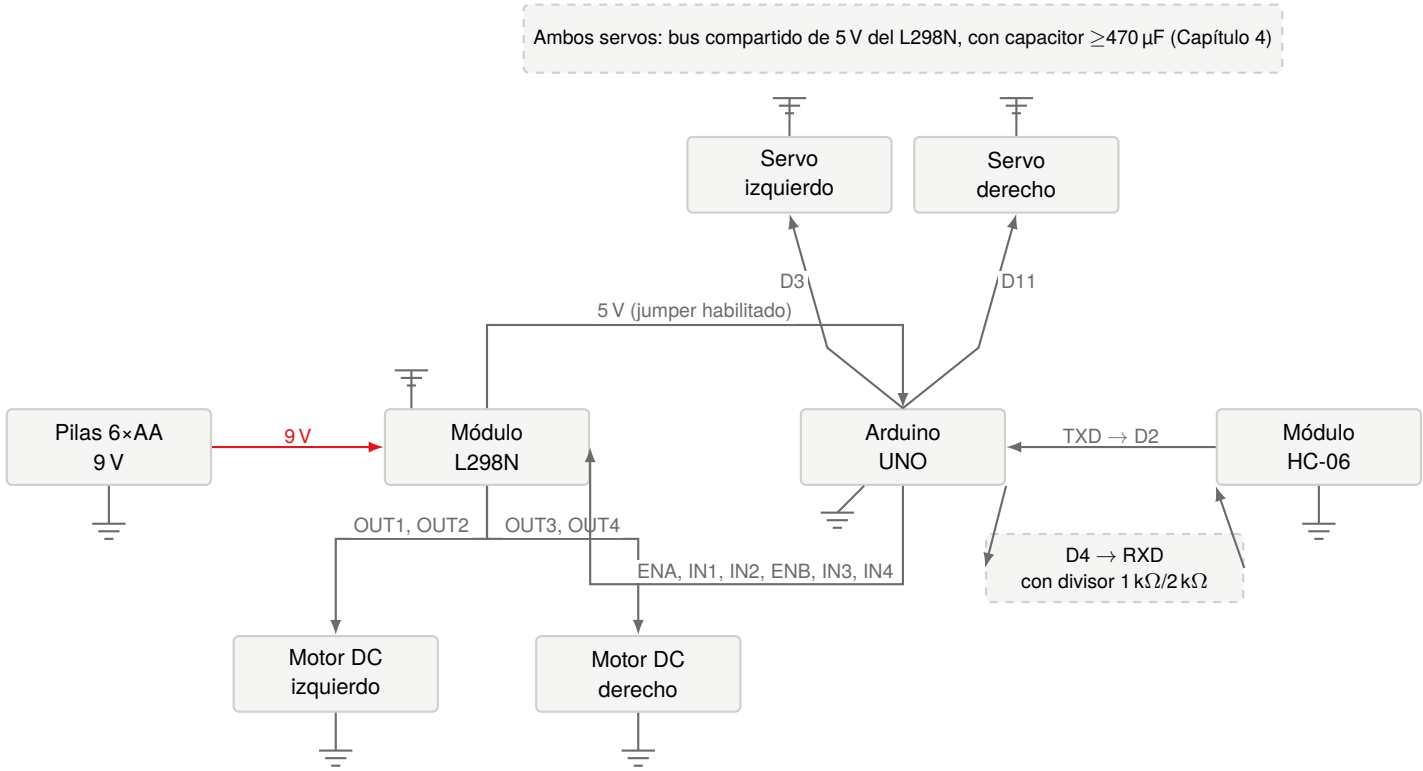
Con los mismos componentes —o agregando uno o dos sensores— puedes construir:

- Un **seguidor de línea**, agregando sensores infrarrojos.
- Un **evasor de obstáculos**, con un sensor ultrasónico.
- Un **brazo robótico**, reutilizando lo que ya sabes de servos.
- Control por **WiFi** en vez de Bluetooth, con un ESP32.
- Un robot con **visión**, agregando una cámara y un poco de procesamiento de imágenes.

Cada uno de esos proyectos parte del mismo lugar donde tú ya estás parado.

Gracias por construir hasta el final. Ahora te toca jugar.

A Diagrama de conexiones



Todos los símbolos de tierra (⊥) están unidos entre sí: GND común obligatorio (Capítulo 4).

B Guía de resolución de problemas

Todas las fallas de esta guía ya aparecieron, una por una, en su capítulo correspondiente. Aquí están juntas, por síntoma, para revisar rápido sin tener que recorrer todo el libro.

Tabla B.1. Impresión 3D (Capítulo 3).

Síntoma	Causa probable	Solución
Las esquinas se levantan de la cama (*warping*)	Mala adherencia o enfriamiento desigual	Usa *brim*, limpia la cama, evita corrientes de aire
Hilos finos entre piezas (*stringing*)	Temperatura muy alta o falta de retracción	Baja 5–10 °C la boquilla, activa retracción
La pieza se despega a mitad de impresión	Cama sucia, sin nivelar, o primera capa muy rápida	Limpia con alcohol isopropílico, nivela, reduce velocidad de la primera capa

Tabla B.2. Electrónica y alimentación (Capítulo 4).

Síntoma	Causa probable	Solución
El Arduino se reinicia al mover un servo	El regulador de 5 V del L298N no entrega corriente suficiente para el Arduino y los servos a la vez	Mide los 5 V con las paletas en movimiento; si caen, usa una fuente regulada de 5 V con corriente suficiente para los servos (GND común) y conserva el capacitor de $\geq$470 µF
El robot se comporta de forma errática o no responde	Falta GND común entre alguna etapa	Verifica continuidad de GND entre pilas, L298N, Arduino y servos
Un motor gira al revés de lo esperado	Cables del motor invertidos en la bornera	Intercambia los dos cables de ese motor en el L298N
El robot pierde fuerza después de un rato de uso	Pilas alcalinas descargándose bajo carga	Cambia a pilas AA recargables NiMH

Tabla B.3. Programación (Capítulo 5).

Síntoma	Causa probable	Solución
Un motor o servo no responde al cargar el código	Cable en el pin equivocado, o alimentación desconectada	Vuelve al Capítulo 4 y revisa esa etapa específica antes de tocar el código
El robot avanza cuando debería retroceder	Motor cableado con polaridad invertida	Intercambia los dos cables de ese motor, no cambies el código
Un servo fuerza o hace ruido al llegar a un extremo	Rango angular no acotado, choca contra el chasis	Ajusta `ANGULO_PATEO` y `ANGULO_REPOSO` para que el brazo nunca llegue al tope mecánico

Tabla B.4. Comunicación (Capítulo 6).

Síntoma	Causa probable	Solución
El celular no encuentra el HC-06	Módulo sin alimentación, o fuera de rango	Verifica el LED del HC-06 (parpadeando = listo para emparejar)
La app no logra conectar al módulo	Emparejamiento Bluetooth del celular no completado antes de abrir la app	Empareja primero desde los ajustes del celular, con el código `1234` o `0000`
Un botón de la app no mueve el robot	El carácter enviado en Bloques no coincide con el protocolo	Revisa el bloque `SendText` contra la Tabla 6.2
El robot sigue moviéndose al soltar el botón	Falta el evento `TouchUp`: el Arduino solo frena al recibir `S`	En cada botón de dirección, agrega un bloque `TouchUp` que envíe `S`

C Archivos descargables

Todos los archivos del libro —el código de Arduino, el modelo 3D del chasis del autor y este mismo libro en PDF— están disponibles para descarga libre en el repositorio:

`https://github.com/Edward3th/UA_Robotica`

Los de esta guía están en la carpeta `guia-1-introduccion-a-la-robotica/`. Para bajar todo de una vez, usa el botón **Code** y luego **Download ZIP** en la página del repositorio; no necesitas una cuenta de GitHub.

Tabla C.1. Archivos de esta guía en el repositorio.

Carpeta o archivo	Contenido
`Introduccion-a-la-Robotica.pdf`	Este libro
`codigo/cap05-01-parpadeo/`	Parpadeo del LED integrado (Capítulo 5)
`codigo/cap05-02-motores/`	Funciones de movimiento con el `L298N` (Capítulo 5)
`codigo/cap05-03-servos/`	Control de las paletas (Capítulo 5)
`codigo/rutina-prueba/`	Rutina de prueba de motores y servos (Capítulo 5)
`codigo/control-bluetooth/`	Programa de control por Bluetooth (Capítulo 6)
`chasis/chasis.stl`	Placa del chasis del autor (Capítulos 2 y 3)
`chasis/guardabarros.stl`	Guardabarros del chasis del autor
`chasis/union-motor-1.stl`	Pieza de unión de los motores al chasis
`chasis/union-motor-2.stl`	Segunda pieza de unión de los motores

NOTA

Cada programa está dentro de una carpeta con su mismo nombre, porque el Arduino IDE así lo exige. Abre el archivo `.ino` desde esa carpeta, sin moverlo ni renombrarlo.

ATENCIÓN

La placa del chasis del autor mide 250×130 mm y no cabe en las camas de 220×220 mm de muchas impresoras. Antes de laminarla, comprueba en Cura que quede dentro del volumen de impresión. Si no cabe, úsala como referencia y diseña tu propia placa, como propone el Capítulo 2.

Bibliografía

[1] Arduino. Arduino Documentation. https://docs.arduino.cc/. Consultado el 2 de octubre de 2026.

[2] Arduino. Servo Library Reference. https://docs.arduino.cc/libraries/servo. Consultado el 2 de octubre de 2026.

[3] Arduino. SoftwareSerial Library Reference. https://docs.arduino.cc/learn/built-in-libraries/software-serial/. Consultado el 2 de octubre de 2026.

[4] Guangzhou HC Information Technology Co., Ltd. HC Serial Bluetooth Products: User Instructional Manual. https://components101.com/sites/default/files/component_datasheet/HC06%20Bluetooth%20Module%20Datasheet.pdf. Manual del fabricante, copia alojada por Components101. Consultado el 2 de octubre de 2026.

[5] MIT. MIT App Inventor Support. https://appinventor.mit.edu/explore/ai2/support/.

[6] Onshape. Onshape Help. https://cad.onshape.com/help/. Consultado el 2 de octubre de 2026.

[7] RoboCup Federation. RoboCup: sitio oficial. https://www.robocup.org/. Consultado el 2 de octubre de 2026.

[8] STMicroelectronics. L298 Dual Full-Bridge Driver. https://www.st.com/resource/en/datasheet/l298.pdf. Hoja de datos del fabricante.

[9] UltiMaker. UltiMaker Cura. https://ultimaker.com/software/ultimaker-cura/. Consultado el 2 de octubre de 2026.